Teoria Współbieżności

Kajetan Frątczak 17.12.2025 r.

Laboratorium 09

Asynchroniczne Przetwarzanie asynchroniczne (wstęp do Node.js)

1. **Treść zadań**

Zadanie 1

1. Zadanie 1a: Zaimplementuj funkcję loop, wg instrukcji w pliku z Rozwiązaniem 3.
2. Zadanie 1b: wykorzystaj funkcję waterfall biblioteki async.

Zadanie 2

Proszę napisać program obliczający liczbę linii we wszystkich plikach tekstowych z danego drzewa katalogów. Do testów proszę wykorzystać zbiór danych Traceroute Data. Program powinien wypisywać liczbę linii w każdym pliku, a na końcu ich globalną sumę. Proszę zmierzyć czas wykonania dwóch wersji programu:

- z synchronicznym (jeden po drugim) przetwarzaniem plików,
- z asynchronicznym (jednoczesnym) przetwarzaniem plików.

Przydatne moduły:

- walkdir -- trawersacja drzewa katalogów
- fs -- operacje na systemie plików (moduł wbudowany)

1. Do obliczania liczby linii w pliku tekstowym proszę wykorzystać następujący fragment kodu:

```
fs.createReadStream(file).on('data', function(chunk) {
        count += chunk.toString('utf8')
        .split(/\r\n|[\n\r\u0085\u2028\u2029]/g)
        .length-1;
    }).on('end', function() {
        console.log(file, count);
    }).on('error', function(err) {
        console.error(err);
    });
```

Fragment ten tworzy strumień i rejestruje trzy funkcje obsługi zdarzeń (wczytanie fragmentu danych, koniec strumienia i wystąpienie błędu). (Zobacz obsługa zdarzeń w Node.js).
W implementacji proszę wykorzystać wzorzec asynchronicznego przetwarzania równoległego opisany tutaj.

2. Argumentacja rozwiązania

Zadanie 1a

W celu zapewnienia sekwencyjnego wykonania asynchronicznych zadań w pętli, nie można użyć standardowej pętli for w sposób bezpośredni, ponieważ spowodowałoby to niemal natychmiastowe uruchomienie wszystkich iteracji równolegle (wyścig). Rozwiązaniem jest zastosowanie wzorca „łańcuchowania obietnic" (ang. Promise Chaining). W zaimplementowanej funkcji loop wykorzystano zmienną pomocniczą p zainicjowaną jako Promise.resolve(). W każdej iteracji pętli do aktualnego łańcucha obietnic dołączana jest kolejna sekwencja zadań za pomocą metody .then(). Dzięki temu każda kolejna iteracja pętli jest rejestrowana do wykonania dopiero po zakończeniu (rozwiązaniu obietnicy) iteracji poprzedniej, co gwarantuje zachowanie kolejności.

Zadanie 1b

W celu wykorzystania biblioteki async i jej funkcji waterfall do wielokrotnego powtórzenia sekwencji, zastosowano podejście rekurencyjne. Funkcja waterfall z biblioteki async służy do wykonania pojedynczej sekwencji zadań, gdzie wynik jednego zadania może być przekazany do następnego. Ponieważ umieszczenie waterfall w pętli for skutkowałoby równoległym uruchomieniem wszystkich cykli, funkcja loop została zaprojektowana tak, aby wywoływać samą siebie. W argumencie funkcji callback (wywoływanej po zakończeniu sekwencji waterfall) następuje dekrementacja licznika m i ponowne wywołanie loop(m-1). Rekurencja kończy się, gdy m <= 0. Gwarantuje to, że nowy cykl zadań rozpocznie się ściśle po zakończeniu poprzedniego.

Zadanie 2

W celu porównania wydajności przetwarzania plików zastosowano dwa podejścia:

1. Podejście synchroniczne: Wykorzystano słowo kluczowe await wewnątrz pętli for...of. Powoduje to, że pętla zdarzeń (Event Loop) wstrzymuje wykonywanie funkcji processSync do momentu zakończenia odczytu i zliczenia linii w bieżącym pliku. Procesor "czeka" na zakończenie operacji I/O przed przejściem do kolejnego pliku.
2. Podejście asynchroniczne: Wykorzystano metodę forEach, która nie czeka na zakończenie asynchronicznych funkcji wewnątrz niej. Uruchomiono proces zliczania linii (countLines) dla wszystkich plików niemal jednocześnie. Do śledzenia postępu wykorzystano licznik pending (inkrementowany przy starcie zadania, dekrementowany przy zdarzeniu end strumienia). Pozwala to systemowi operacyjnemu i bibliotece libuv (pod spodem Node.js) na równoległe obsługiwanie żądań odczytu z dysku, co maksymalizuje przepustowość.

3. Rozwiązanie

Zadanie 1a

```
function printAsync(s, cb) {
  var delay = Math.floor((Math.random() * 1000) + 500);
  setTimeout(function () {
    console.log(s);
    if (cb) cb();
  }, delay);
}
```

```javascript
function task(n) {
  return new Promise((resolve, reject) => {
    printAsync(n, function () {
      resolve(n);
    });
  });
}

// 'then' returns a new Promise, therefore we can chain another 'then'.
// In this case 'task(x)' directly returns a Promise object, however
// 'then' could also return a value in which case it would be wrapped
// in a Promise that would be automatically resolved with that value.

/*
** Zadanie:
** Napisz funkcje loop(m), ktora powoduje wykonanie powyzszej
** sekwencji zadan m razy.
**
*/

function loop(m) {
    let p = Promise.resolve();

    for (let i = 0; i < m; i++) {
        p = p.then(() => {
            return task(1)
                .then((n) => {
                    console.log('task', n, 'done');
                    return task(2);
                })
                .then((n) => {
                    console.log('task', n, 'done');
                    return task(3);
                })
                .then((n) => {
                    console.log('task', n, 'done');
                    console.log('--- sekwencja zakończona ---\n');
                });
        });
    }

    p.then(() => {
        console.log("Koniec pętli loop");
    });
}

loop(4);
```

Zadanie 1b

```javascript
const async = require('async');

function printAsync(s, cb) {
  var delay = Math.floor((Math.random() * 1000) + 500);
  setTimeout(function () {
    console.log(s);
    if (cb) cb();
  }, delay);
}
```

```
function task1(cb) {
    printAsync("1", () => {
        cb(null);
    });
}

function task2(cb) {
    printAsync("2", () => {
        cb(null);
    });
}

function task3(cb) {
    printAsync("3", () => {
        cb(null);
    });
}

function loop(m) {
    if (m <= 0) {
        console.log('Koniec pętli loop');
        return;
    }

    async.waterfall([
        task1,
        task2,
        task3
    ], function (err, result) {
        if (err) {
            console.error(err);
        } else {
            console.log('--- sekwencja zakończona ---\n');
            loop(m - 1);
        }
    });
}
loop(4);
```

Zadanie 2

```
const fs = require('fs');
const walkdir = require('walkdir');
const path = require('path');
const DATA_DIR = './data';
const countLines = (file) => {
    return new Promise((resolve, reject) => {
        let count = 0;
        fs.createReadStream(file)
            .on('data', function(chunk) {
                count += chunk.toString('utf8')
                .split(/\r\n|[\n\r\u0085\u2028\u2029]/g)
                .length-1;
            })
            .on('end', function() {
                resolve(count);
            })
            .on('error', reject);
    });
};
```

```javascript
// --- Wersja 1: Synchroniczne przetwarzanie (jeden po drugim) ---
async function processSync() {
    console.time('Sync');
    let totalLines = 0;
    const files = walkdir.sync(DATA_DIR);

    for (const file of files) {
        if (fs.lstatSync(file).isFile()) {
            const lines = await countLines(file);
            console.log(file, lines);
            totalLines += lines;
        }
    }
    console.log('Total Sync:', totalLines);
    console.timeEnd('Sync');
}

// --- Wersja 2: Asynchroniczne przetwarzanie (równoległe) ---
function processAsync() {
    console.time('Async');
    let totalLines = 0;
    let paths = walkdir.sync(DATA_DIR);
    let pending = 0;

    let files = paths.filter(p => fs.lstatSync(p).isFile());

    if(files.length === 0) {
         console.timeEnd('Async');
         return;
    }

    files.forEach(file => {
        pending++;
        countLines(file).then(lines => {
            console.log(file, lines);
            totalLines += lines;
            pending--;
            if (pending === 0) {
                console.log('Total Async:', totalLines);
                console.timeEnd('Async');
            }
        });
    });
}

processSync();
//processAsync();
```

4. Wykresy, tabele, wyniki

Zadanie 1a

1
task 1 done
2
task 2 done
3
task 3 done
--- sekwencja zakończona ---

1
task 1 done
2
task 2 done
3
task 3 done
--- sekwencja zakończona ---

1
task 1 done
2
task 2 done
3
task 3 done
--- sekwencja zakończona ---

1
task 1 done
2
task 2 done
3
task 3 done
--- sekwencja zakończona ---
Koniec pętli loop

Zadanie 1b

1
2
3
--- sekwencja zakończona ---

1
2
3
--- sekwencja zakończona ---

1
2
3
--- sekwencja zakończona ---

1
2
3
--- sekwencja zakończona ---
Koniec pętli loop

Zadanie 2

Total Async: 61790
Async: 411.61ms

Total Sync: 61790
Sync: 929.752ms

5. Wnioski

- **Wydajność operacji I/O:** Przetwarzanie asynchroniczne okazało się ponad **2-krotnie szybsze** (411 ms vs 929 ms) od przetwarzania synchronicznego. Wynika to z nieblokującej natury Node.js. W wersji asynchronicznej, w czasie oczekiwania na dane z dysku dla jednego pliku, wątek Node.js może inicjować odczyt kolejnych plików. W wersji synchronicznej czas ten jest tracony na bezczynne oczekiwanie.
- **Zarządzanie przepływem:** Implementacja sekwencyjnego wykonywania zadań asynchronicznych (Zadanie 1) wymaga świadomego użycia odpowiednich wzorców projektowych. Użycie obietnic (`Promise chaining`) lub bibliotek pomocniczych (`async.waterfall` z rekurencją) jest niezbędne, aby uniknąć zjawiska "wyścigu" i zapewnić przewidywalną kolejność zdarzeń.
- **Złożoność kodu:** Kod asynchroniczny (wykorzystujący strumienie i zdarzenia) jest trudniejszy w implementacji i debugowaniu niż kod wykorzystujący składnię `async/await` w sposób sekwencyjny. Istnieje jednak wyraźny kompromis (trade-off) pomiędzy prostotą kodu a jego wydajnością przy dużej liczbie operacji wejścia-wyjścia.

6. Bibliografia

https://nodejs.org/en
https://www.npmjs.com/
https://github.com/creationix/howtonode.org/tree/master/articles
https://kikobeats.com/synchronously-asynchronous